Contrato de Interface — Mentora → Arquiteto de Conteúdo

Especificação técnica do pacote de post Femba

Versão: 1.0 · **Data**: 23/10/2026

Partes: Mentora de Posts (produção) · Arquiteto de Conteúdo (publicação)

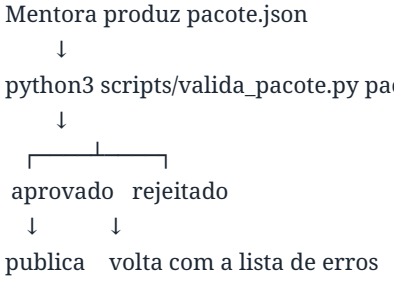
Implementação: scripts/valida_pacote.py · Schema: artifacts/schemas/pacote-post-v1.json

Testes: scripts/testa_validador.py — 12/12 passando

1. Por que existe um contrato

Sem contrato, o pacote chega "quase certo" e a falha aparece no meio da publicação: a URL não abre, o carrossel tem 1 arte, a legenda passa de 2.200 caracteres, a data já passou. Nesse ponto, publicar é pior que não publicar — o Instagram não tem "desfazer" para um post que saiu errado.

A regra central: o validador roda **antes** de qualquer chamada à API. Se algo falhar, o pacote é **rejeitado por inteiro** e volta para produção. Nunca publica parcialmente.



2. O pacote — campos obrigatórios

O JSON abaixo é o mínimo válido. Todos os campos são obrigatórios salvo os marcados *(opcional)*.

```
{
  "id": "POST-2026-11-09-01",
  "versao_contrato": "1.0",
```

```
  "data_publicacao": "2026-11-09T12:00:00-03:00",
  "canais": ["instagram", "facebook"],
  "formato": "carrossel",
  "campanha": "Escolas & Creches",
  "segmento": "escolas",
  "palavra_chave": "ESCOLA",
  "produto": null,
  "artes": [
    {"ordem": 1, "url": "https://.../01_capa.jpg", "proporcao": "1:1", "alt": "Capa 3 erros"},
    {"ordem": 2, "url": "https://.../02_erro1.jpg", "proporcao": "1:1", "alt": "Erro 1"},
    {"ordem": 3, "url": "https://.../03_erro2.jpg", "proporcao": "1:1", "alt": "Erro 2"},
    {"ordem": 4, "url": "https://.../04_erro3.jpg", "proporcao": "1:1", "alt": "Erro 3"},
    {"ordem": 5, "url": "https://.../05_cta.jpg", "proporcao": "1:1", "alt": "CTA"}
  ],
  "legenda": "texto completo, com o bloco fixo de serviço já anexado...",
  "comentario_fixado": "Orçamento em 24h: (11) 3931-3180 ",
  "hashtags": ["#escolas", "#creche", "#limpeza", "#femba"],
  "qa": {
    "formato_ok": true,
    "logo_oficial": true,
    "cores_paleta": true,
    "contraste_aa": true,
    "texto_sem_corte": true,
    "icones_sem_colisao": true,
    "rodape_com_respiro": true,
    "legenda_chars": 742,
    "urls_testadas": ["https://.../01_capa.jpg"]
  },
  "aprovacao": {"por": "Igor", "em": "2026-11-06T18:00:00-03:00"}
}
```


2.1 Tabela de campos

Campo	Tipo	Regra
id	texto	Padrão POST-AAAA-MM-DD-NN. Único por post — evita publicação duplicada
versao_contrato	texto	Exatamente "1.0". Versão diferente é rejeitada
data_publicacao	texto	ISO 8601 **com fuso `-03:00`**. Não pode estar no passado
canais	lista	Ao menos 1 de: instagram, facebook, instagram_story
formato	texto	imagem, carrossel, story ou reels
campanha	texto	3 a 60 caracteres
palavra_chave	texto	2 a 20 letras MAIÚSCULAS. Use NENHUMA se não houver

		rastreio
segmento ***(opcional)***	texto	escolas, creches, condominios, food_service, escritorios, clinicas, industrias, geral
produto ***(opcional)***	texto ou null	Nome comercial, até 80 caracteres
artes	lista	1 a 10 itens (ver 2.2)
legenda	texto	40 a 2.200 caracteres, **com o bloco fixo já dentro**
comentario_fixado	texto ou null	Obrigatório em carrossel
hashtags	lista	4 a 6 itens, e todas devem estar na legenda
qa	objeto	Todas as flags true (ver seção 4)
aprovacao	objeto	por + em (timestamp ISO)

2.2 Estrutura de cada arte

Campo	Regra
ordem	Inteiro 1..N **sem buracos**. Para carrossel, 1..5, nunca 1,3,5
url	**HTTPS obrigatório**. HTTP simples é rejeitado
proporcao	1:1, 4:5, 9:16 ou 16:9 — precisa ser compatível com o formato
alt ***(opcional)***	Até 120 caracteres
legenda_alt ***(opcional)***	Até 300 caracteres

Proporções aceitas por formato:

Formato	Proporções válidas
imagem	1:1, 4:5
carrossel	1:1, 4:5 — **todas as artes iguais**
story	9:16
reels	9:16

3. As três camadas de validação

Camada 1 — Schema (estrutura)

Verifica tipos, campos obrigatórios, formatos de texto, listas fechadas (enum) e campos não previstos.

Campo não previsto no contrato é rejeitado — isso impede que combinem "preco": "R$ 51,90" sem que ninguém tenha confirmado o valor.

Camada 2 — Servidor (a arte realmente existe)

O Instagram **baixa a imagem pela URL**. Se ela não abrir no servidor, não publica. Então o validador abre cada URL e exige:

Verificação	Critério
Status HTTP	**200** exato
Content-Type	precisa começar com image/
Tamanho mínimo	8 KB (abaixo disso é placeholder)
Tamanho máximo	8 MB (acima o Instagram recusa)
Alerta de peso	acima de 800 KB emite aviso — comprimir melhora o carregamento

Camada 3 — Regras (coerência entre os campos)

O schema não sabe se o carrossel faz sentido. Esta camada sabe:

#	Regra	Por que
1	Carrossel exige 2 a 10 artes; imagem e story exigem exatamente 1	O Instagram rejeita fora disso
2	ordem contígua 1..N	Ordem com buraco gera slide faltando
3	Proporção compatível e homogênea no carrossel	O IG corta todas as artes pela proporção da primeira
4	Legenda ≤ 2.200 caracteres	Limite da Meta
5	Legenda contém a assinatura da marca	Sem bloco fixo, o lead não sabe como comprar
6	Hashtags declaradas aparecem na legenda	Declarar e não usar é erro silencioso
7	Máximo 30 hashtags	Limite da Meta
8	Sem palavras proibidas	Ver 3.1
9	Data não está no passado	Evita publicar com data errada
10	Palavra-chave aparece na legenda	Sem ela, o CRM não atribui o lead
11	Flags de QA todas verdadeiras	Quem assina, responde
12	qa.urls_testadas cobre todas as artes	Não se pode declarar testado o que não foi
13	Indica quando o canal é instagram_story	A API não publica stories — exige ação manual

3.1 Assinatura da marca (obrigatória na legenda)

A legenda precisa conter **estes quatro elementos**:

Elemento	Texto procurado
WhatsApp	(11) 3931 3180

Telefone	(11) 2925 3180
Prazo	24h
Pedido mínimo	R$200,00

Colando o bloco fixo da seção 8.3 do Manual de Produção Visual, os quatro entram automaticamente.

3.2 Palavras proibidas (rejeição automática)

garantido · garantia total · o melhor do mercado · melhor do brasil · exclusivo · imperdível · único no mercado · viral · viralizar · 100% aprovado · resultado garantido

Motivo: promessa que não se pode comprovar expõe a empresa. Se a intenção é dizer que o produto é bom, diga **o que ele faz**.

4. O bloco qa — a assinatura de quem produziu

Sete flags booleanas. Todas precisam ser true. Se uma for false, o pacote é rejeitado — **mesmo que o resto esteja perfeito**.

Flag	Confirma que
formato_ok	A arte está em 1080×1080 (ou 1080×1350 / 1080×1920 conforme o formato)
logo_oficial	É o arquivo femba_logo.png colado como imagem — não redesenhado, não gerado por IA
cores_paleta	Só #1669a6, #94c41c, #1c2733, #55606b, #f0f2f4, #102a43
contraste_aa	Toda combinação texto/fundo ≥ 4,5:1
texto_sem_corte	Nenhum texto encosta na borda nem é cortado por elemento
icones_sem_colisao	Gap ≥ 14 px entre ícone e texto
rodape_com_respiro	Padding ≥ 12 px no topo e na base da barra

Mais dois campos de fato:

- legenda_chars — número exato de caracteres da legenda (o validador confere)
- urls_testadas — lista de todas as URLs verificadas

Estas sete flags não são burocracia: cada uma nasceu de um defeito que chegou perto de ser publicado. Marcar true sem ter feito é o único jeito de furar o contrato — e o custo recai sobre a marca.

5. Como rodar

5.1 Linha de comando

```
# validação completa (recomendado — testa as URLs de verdade)
python3 scripts/valida_pacote.py caminho/pacote.json

# sem testar URL (só durante a produção, NUNCA antes de publicar)
python3 scripts/valida_pacote.py caminho/pacote.json --offline

# saída em JSON para automação
python3 scripts/valida_pacote.py caminho/pacote.json --json
```

5.2 Códigos de retorno

Exit	Significado	Ação
0	Aprovado	Pode publicar
1	Rejeitado	Devolver à produção com a lista de erros
2	Erro de uso	Arquivo inexistente ou JSON inválido

5.3 O que a saída parece

Aprovado:

```
======================================================================
 VALIDAÇÃO DE PACOTE — POST-2026-10-29-01
======================================================================
 ok    camada 1 (schema): estrutura válida
 ok    agenda ok: 29/10/2026 12:00 BRT
 ok    url ok: arte_rambo_max_5l.png (1421 KB, image/png)
 AVISO  [servidor] arte_rambo_max_5l.png com 1421 KB (guia da marca: <781 KB)
----------------------------------------------------------------------
 RESULTADO: APROVADO PARA PUBLICAÇÃO
======================================================================
```

Rejeitado:

```
 ERRO   [regra] legenda sem assinatura da marca: falta ['24h', 'R$200,00']
 ERRO   [regra] palavras proibidas na legenda: ['garantido']
 ERRO   [servidor] https://.../nao_existe.png — HTTP 404
----------------------------------------------------------------------
 RESULTADO: REJEITADO — 3 erro(s) · 0 aviso(s)
 AÇÃO: devolver o pacote à produção. Nada será publicado.
======================================================================
```

6. O que o Arquiteto faz com um pacote aprovado

1. valida_pacote.py → exit 0
2. checa cota do Instagram (content_publishing_limit)
3. checa colisão na agenda (máx 2 posts/dia, ≥6h de intervalo)
4. publica cada arte (container → publish)
5. publica o pai do carrossel, se for o caso
6. publica no Facebook com a mesma arte e legenda
7. registra IDs e permalinks
8. atualiza o calendário

Ponto de atenção real: a Meta documenta números diferentes para o limite de publicação por 24h — **100** na seção "Rate Limit" do guia de publicação e **50** na seção de limitações do carrossel. A recomendação oficial é **consultar o endpoint `content_publishing_limit` em vez de fixar o número**. O código do Arquiteto deve fazer isso: perguntar à API, não assumir.

7. O que acontece quando o pacote é rejeitado

Situação	Quem resolve	O que fazer
Erro de schema (campo faltando, tipo errado)	Mentora	Corrigir o JSON e revalidar
Erro de servidor (URL 404, arquivo pequeno)	Mentora	Rehospedar a arte e testar no navegador anônimo
Erro de regra (legenda, hashtags, data)	Mentora	Ajustar o conteúdo e revalidar
QA falso (flag marcada sem ter feito)	Mentora	Refazer a arte e medir de verdade
Pacote NENHUMA palavra-chave em campanha que exige rastreio	Arquiteto	Definir a palavra e pedir reemissão

Nunca corrija o pacote para "passar no validador". O validador existe para barrar o defeito, não para ser contornado.

8. Regra de escalonamento

Se um defeito resistir a **duas tentativas** de correção, pare e chame o Designer com:

1. O arquivo da arte

2. A **região exata** do problema (coordenadas ou descrição precisa)

3. A medida esperada vs a encontrada

Não gaste uma terceira rodada no mesmo erro sem informação nova. E lembre da lição do próprio Designer: **medir a região apontada**, nunca varrer a imagem inteira e concluir "não existe".

9. Versionamento do contrato

- versao_contrato muda **só** quando houver quebra de compatibilidade (campo removido ou obrigatório novo)
- Adicionar campo **opcional** não muda a versão
- Toda mudança é registrada aqui, com data

Versão	Data	Mudança
1.0	23/10/2026	Versão inicial

10. Checklist de aceite antes de entregar o pacote

Rode mentalmente antes de rodar o script — economiza uma rodada:

- [ ] Todas as artes estão em URL **HTTPS** que abre no navegador anônimo
- [ ] Cada arte no formato certo (1080×1080, 1080×1350 ou 1080×1920)
- [ ] Carrossel: 2 a 10 artes, **todas na mesma proporção**, ordem 1..N
- [ ] Legenda com o **bloco fixo de serviço colado** (garante os 4 elementos da assinatura)
- [ ] Palavra-chave aparece na legenda
- [ ] Hashtags declaradas são as mesmas que estão na legenda
- [ ] Data com fuso -03:00 e no futuro
- [ ] As 7 flags de QA marcadas **depois** de medir, não por otimismo
- [ ] legenda_chars é o número real
- [ ] Aprovação registrada com nome e timestamp

11. Um aviso honesto sobre o limite deste contrato

O validador garante que o pacote está **estruturalmente correto e publicável**. Ele não sabe se a arte é **boa**: se o gancho prende, se a hierarquia funciona, se a foto do produto está bonita.

Isso continua sendo julgamento humano — seu e do Arquiteto. O contrato protege contra o erro objetivo (URL morta, texto cortado, logo trocado, promessa ilegal). O erro subjetivo continua precisando de olho.

Por isso o qa existe: ele não substitui a revisão, ele **registra** que a revisão aconteceu.

Implementação: scripts/valida_pacote.py · Schema: artifacts/schemas/pacote-post-v1.json · Testes: scripts/testa_validador.py (12/12) · Complementa: artifacts/Manual_Producao_Visual_Femba_Mentora.docx